

Object Oriented Programming (CS1004)

Sessional-11 Exam

Date:

Total Time (Hrs): 1

Course Instructor(s)

Total Marks: 30

Ms. Sumaiyah Zahid, Ms. Sobia Iftikhar, Ms. Abeer Gauher, Mr.
Ghulam Qadir, Mr. Ubaidullah, Mr. Talha Shahid, Mr. Aashir
Mehboob, Mr. Saif Ur Rehman, Mr. Syed Ahmed Khan

Total Questions: 3

Roll No

Section

Student Signature

Do not write below this line.

Question # 1:

Part A. Provide 2 to 3 lines of conceptual reasoning

- a. Base argument will accept (10)
- b. Prefix (++obj): `operator++()` Postfix (obj++): `operator++(int)` (dummy int parameter)
- c. No, friendship is not mutual. Friendship is not inherited.

Part B: Predict the output of following

- a. 14;
- b. Base C
Derived C
Derived D
Base D

Scenario:

Question 02:

```
class Drone {
```

```
protected:
```

```
    int id;
```

```
    int batteryLife;
```

```
    bool connected;
```

```
    bool powerOn;
```

```
public:
```

```
    Drone(int i = 0, int b = 0, bool c = false, bool p = false)
```

```
    : id(i), batteryLife(b), connected(c), powerOn(p) {}
```

```

bool isturnOn() { return powerOn; }

bool isConnected() { return connected; }

int getBatteryLife() const { return batteryLife; } // FIX

virtual void performTask() {

    cout << "Base Drone Task\n";

}

friend class DroneControlCenter;

friend istream& operator>>(istream& in, Drone& d);

bool operator>(const Drone& other) {

    return batteryLife > other.batteryLife;

}

};

class DeliveryDrone : public Drone {

private:

    int packageCount;

    void deliverPackages(int numPackages) {

        packageCount += numPackages;

        cout << "Delivered " << numPackages

            << " packages. Total: " << packageCount << endl;

    }

public:

    DeliveryDrone(int i = 0, int b = 0, bool c = false, bool p = false, int pc = 0)

        : Drone(i, b, c, p), packageCount(pc) {}

    void performTask() override {

        if (powerOn && connected) {

            deliverPackages(1);

        } else {

```

```

        cout << "DeliveryDrone not ready\n";
    }
}

friend istream& operator>>(istream& in, DeliveryDrone& d);
};

class SurveillanceDrone : public Drone {
private:
    string area;
public:
    SurveillanceDrone(int i = 0, int b = 0, bool c = false, bool p = false, string a = "")
        : Drone(i, b, c, p), area(a) {}

    void monitorArea(string areaName) {
        cout << "Monitoring: " << areaName << endl;
    }

    void performTask() override {
        if (powerOn && connected) {
            monitorArea(area);
        } else {
            cout << "SurveillanceDrone not ready\n";
        }
    }

    void operator+=(DeliveryDrone& d) {
        batteryLife += d.getBatteryLife(); // ✓ FIX
        if (batteryLife > 100)
            batteryLife = 100;
    }
};

class DroneOperator {
private:

```

```

    Drone** drones;

    int capacity;

    int count;

public:
    DroneOperator(int cap) {
        capacity = cap;

        count = 0;

        drones = new Drone*[capacity];
    }

    void assignDrone(Drone* d) {
        if (count < capacity)
            drones[count++] = d;
    }

    void performAllTasks() {
        for (int i = 0; i < count; i++) {
            drones[i]->performTask(); // polymorphism
        }
    }

    ~DroneOperator() {
        delete[] drones;
    }
};

```

Question 03:

```

class DroneControlCenter {
public:
    void emergencyShutdown(Drone* d) {

```

```

        d->powerOn = false;

        cout << "Drone shutdown\n";
    }

    void boostBattery(Drone& d, int amount) {
        d.batteryLife += amount;
        if (d.batteryLife > 100)
            d.batteryLife = 100;
    }
};

istream& operator>>(istream& in, Drone& d) {
    cout << "Enter id, battery, connected(1/0), power(1/0): ";
    in >> d.id >> d.batteryLife >> d.connected >> d.powerOn;
    return in;
}

istream& operator>>(istream& in, DeliveryDrone& d) {
    cout << "Enter id, battery, connected(1/0), power(1/0), packages: ";
    in >> d.id >> d.batteryLife >> d.connected >> d.powerOn >> d.packageCount;
    return in;
}

int main() {
    DeliveryDrone d1, d2;

    cin >> d1;
    cin >> d2;

    SurveillanceDrone s1(2, 70, false, false, "Sector A");

```

```
if (d1 > d2)
    cout << "Drone 1 has more battery\n";
else
    cout << "Drone 2 has more battery\n";

s1 += d1;

DroneOperator op(3);
op.assignDrone(&d1);
op.assignDrone(&d2);
op.assignDrone(&s1);

op.performAllTasks();

return 0;
}
```